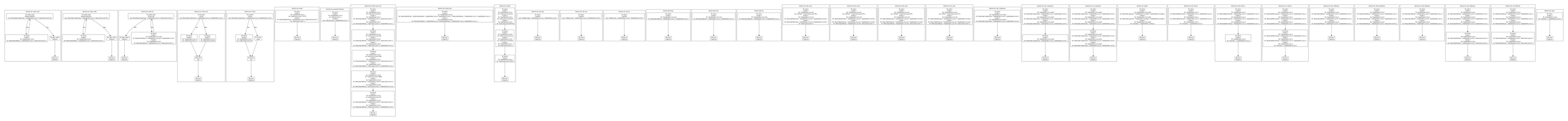

function test_repeat_until
B2 (while.cond)
RepeatCond(until)
expr: BinaryOp(CompareExpr) { Nop(Identifier) [var:i] | Nop(Literal) [const:10] }
false
true
B1 (block)
Assign(=)
lhs: Nop(Identifier) [var:i]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:i] | Nop(Literal) [const:1] }
B3 (after_while)
Nop(exit)
B0 (exit)
Nop(exit)
function test_repeat_while
B2 (while.cond)
RepeatCond(while)
expr: BinaryOp(CompareExpr) { Nop(Identifier) [var:i] | Nop(Literal) [const:10] }
true
false
B1 (block)
Assign(=)
lhs: Nop(Identifier) [var:i]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:i] | Nop(Literal) [const:1] }
B3 (after_while)
Nop(exit)
B0 (exit)
Nop(exit)
function test_while_do
B1 (while.cond)
WhileCond
expr: BinaryOp(CompareExpr) { Nop(Identifier) [var:i] | Nop(Literal) [const:0] }
false
true
B2 (after_while)
Nop(exit)
B3 (block)
Assign(=)
lhs: Nop(Identifier) [var:sum]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:sum] | Nop(Identifier) [var:i] }
Assign(=)
lhs: Nop(Identifier) [var:i]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:i] | Nop(Literal) [const:1] }
B0 (exit)
Nop(exit)
function test_if_then_else
B1 (if.cond)
IfCond
expr: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
true
false
B2 (block)
Assign(=)
lhs: Nop(Identifier) [var:c]
rhs: Nop(Literal) [const:1]
B3 (block)
Assign(=)
lhs: Nop(Identifier) [var:c]
rhs: Nop(Literal) [const:0]
B4 (if.join)
join
B0 (exit)
Nop(exit)
function test_if_then
B1 (if.cond)
IfCond
expr: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
true
false
B2 (block)
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: Nop(Literal) [const:10]
B3 (if.else)
empty
B4 (if.join)
join
B0 (exit)
Nop(exit)
function test_strings
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:i]
rhs: Nop(Literal) [const:"12312"]
Assign(=)
lhs: BinaryOp(IndexExpr) { Nop(Identifier) [var:i] | Nop(Literal) [const:5] }
rhs: Nop(Literal) [const:'1']
B0 (exit)
Nop(exit)
function test_generated_functions
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:i]
rhs: Call(read_byte) { }
ExprStmt
expr: Call(send_byte) { Nop(Identifier) [var:i] }
B0 (exit)
Nop(exit)
function test_literals_auto_type
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:i]
rhs: Nop(Literal) [const:42]
Assign(=)
lhs: Nop(Identifier) [var:i]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:i] | Nop(Literal) [const:1] }
Assign(=)
lhs: Nop(Identifier) [var:i]
rhs: BinaryOp(AddExpr) { Nop(Literal) [const:1] | Nop(Identifier) [var:i] }
B2 (block)
Assign(=)
lhs: Nop(Identifier) [var:u]
rhs: Nop(Literal) [const:100]
Assign(=)
lhs: Nop(Identifier) [var:u]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:u] | Nop(Literal) [const:1] }
Assign(=)
lhs: Nop(Identifier) [var:u]
rhs: BinaryOp(AddExpr) { Nop(Literal) [const:1] | Nop(Identifier) [var:u] }
B3 (block)
Assign(=)
lhs: Nop(Identifier) [var:l]
rhs: Nop(Literal) [const:1000]
Assign(=)
lhs: Nop(Identifier) [var:l]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:l] | Nop(Literal) [const:1] }
Assign(=)
lhs: Nop(Identifier) [var:l]
rhs: BinaryOp(AddExpr) { Nop(Literal) [const:1] | Nop(Identifier) [var:l] }
B4 (block)
Assign(=)
lhs: Nop(Identifier) [var:ul]
rhs: Nop(Literal) [const:10000]
Assign(=)
lhs: Nop(Identifier) [var:ul]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:ul] | Nop(Literal) [const:1] }
Assign(=)
lhs: Nop(Identifier) [var:ul]
rhs: BinaryOp(AddExpr) { Nop(Literal) [const:1] | Nop(Identifier) [var:ul] }
B5 (block)
Assign(=)
lhs: Nop(Identifier) [var:b]
rhs: Nop(Literal) [const:255]
Assign(=)
lhs: Nop(Identifier) [var:b]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:b] | Nop(Literal) [const:1] }
Assign(=)
lhs: Nop(Identifier) [var:b]
rhs: BinaryOp(AddExpr) { Nop(Literal) [const:1] | Nop(Identifier) [var:b] }
B0 (exit)
Nop(exit)
function test_nested_expr
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:d]
rhs: BinaryOp(MulExpr) { BinaryOp(AddExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] } | BinaryOp(AddExpr) { Nop(Identifier) [var:c] | Nop(Identifier) [var:a] } }
Assign(=)
lhs: Nop(Identifier) [var:d]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:a] | BinaryOp(MulExpr) { Nop(Identifier) [var:b] | Nop(Identifier) [var:c] } }
B0 (exit)
Nop(exit)
function test_literals
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:i]
rhs: Nop(Literal) [const:42]
Assign(=)
lhs: Nop(Identifier) [var:i]
rhs: Nop(Literal) [const:0xFF]
Assign(=)
lhs: Nop(Identifier) [var:i]
rhs: Nop(Literal) [const:0b1010]
B2 (block)
Assign(=)
lhs: Nop(Identifier) [var:b]
rhs: Nop(Literal) [const:true]
Assign(=)
lhs: Nop(Identifier) [var:b]
rhs: Nop(Literal) [const:false]
Assign(=)
lhs: Nop(Identifier) [var:c]
rhs: Nop(Literal) [const:'x']
B3 (block)
Assign(=)
lhs: Nop(Identifier) [var:s]
rhs: Nop(Literal) [const:"hello"]
B0 (exit)
Nop(exit)
function test_call_long
B1 (entry)
ExprStmt
expr: Call(add_longs) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
B0 (exit)
Nop(exit)
function test_call_uint
B1 (entry)
ExprStmt
expr: Call(add_uints) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
B0 (exit)
Nop(exit)
function test_call_int
B1 (entry)
ExprStmt
expr: Call(add_ints) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
B0 (exit)
Nop(exit)
function add_longs
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:sum]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:x] | Nop(Identifier) [var:y] }
B0 (exit)
Nop(exit)
function add_uints
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:sum]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:x] | Nop(Identifier) [var:y] }
B0 (exit)
Nop(exit)
function add_ints
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:sum]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:x] | Nop(Identifier) [var:y] }
B0 (exit)
Nop(exit)
function test_char_array
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:arr]
rhs: Call(char) { Nop(Literal) [const:100] }
Assign(=)
lhs: Nop(Identifier) [var:val]
rhs: BinaryOp(IndexExpr) { Nop(Identifier) [var:arr] | Nop(Identifier) [var:idx] }
Assign(=)
lhs: BinaryOp(IndexExpr) { Nop(Identifier) [var:arr] | Nop(Identifier) [var:idx] }
rhs: Nop(Literal) [const:'x']
B0 (exit)
Nop(exit)
function test_byte_array
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:arr]
rhs: Call(byte) { Nop(Literal) [const:256] }
Assign(=)
lhs: Nop(Identifier) [var:val]
rhs: BinaryOp(IndexExpr) { Nop(Identifier) [var:arr] | Nop(Identifier) [var:idx] }
Assign(=)
lhs: BinaryOp(IndexExpr) { Nop(Identifier) [var:arr] | Nop(Identifier) [var:idx] }
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:val] | Nop(Literal) [const:1] }
B0 (exit)
Nop(exit)
function test_uint_array
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:arr]
rhs: Call(uint) { Nop(Literal) [const:20] }
Assign(=)
lhs: Nop(Identifier) [var:val]
rhs: BinaryOp(IndexExpr) { Nop(Identifier) [var:arr] | Nop(Identifier) [var:idx] }
Assign(=)
lhs: BinaryOp(IndexExpr) { Nop(Identifier) [var:arr] | Nop(Identifier) [var:idx] }
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:val] | Nop(Literal) [const:1] }
B0 (exit)
Nop(exit)
function test_int_array
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:arr]
rhs: Call(int) { Nop(Literal) [const:10] }
Assign(=)
lhs: Nop(Identifier) [var:val]
rhs: BinaryOp(IndexExpr) { Nop(Identifier) [var:arr] | Nop(Identifier) [var:idx] }
Assign(=)
lhs: BinaryOp(IndexExpr) { Nop(Identifier) [var:arr] | Nop(Identifier) [var:idx] }
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:val] | Nop(Literal) [const:1] }
B0 (exit)
Nop(exit)
function test_char_comparison
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:result]
rhs: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
Assign(=)
lhs: Nop(Identifier) [var:result]
rhs: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
B0 (exit)
Nop(exit)
function test_uint_comparison
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:result]
rhs: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
Assign(=)
lhs: Nop(Identifier) [var:result]
rhs: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
Assign(=)
lhs: Nop(Identifier) [var:result]
rhs: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
B2 (block)
Assign(=)
lhs: Nop(Identifier) [var:result]
rhs: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
B0 (exit)
Nop(exit)
function test_int_comparison
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:result]
rhs: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
Assign(=)
lhs: Nop(Identifier) [var:result]
rhs: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
Assign(=)
lhs: Nop(Identifier) [var:result]
rhs: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
B2 (block)
Assign(=)
lhs: Nop(Identifier) [var:result]
rhs: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
Assign(=)
lhs: Nop(Identifier) [var:result]
rhs: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
Assign(=)
lhs: Nop(Identifier) [var:result]
rhs: BinaryOp(CompareExpr) { Nop(Identifier) [var:a] | Nop(Identifier) [var:b] }
B0 (exit)
Nop(exit)
function test_logical
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:result]
rhs: BinaryOp(LogicExpr) { Nop(Identifier) [var:flag1] | Nop(Identifier) [var:flag2] }
Assign(=)
lhs: Nop(Identifier) [var:result]
rhs: BinaryOp(LogicExpr) { Nop(Identifier) [var:flag1] | Nop(Identifier) [var:flag2] }
Assign(=)
lhs: Nop(Identifier) [var:result]
rhs: UnaryOp(!) { Nop(Identifier) [var:flag1] }
B0 (exit)
Nop(exit)
function test_byte_bitwise
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:x]
rhs: BinaryOp(BitwiseExpr) { Nop(Identifier) [var:y] | Nop(Identifier) [var:z] }
Assign(=)
lhs: Nop(Identifier) [var:x]
rhs: BinaryOp(BitwiseExpr) { Nop(Identifier) [var:y] | Nop(Identifier) [var:z] }
Assign(=)
lhs: Nop(Identifier) [var:x]
rhs: UnaryOp(~) { Nop(Identifier) [var:y] }
B0 (exit)
Nop(exit)
function test_uint_bitwise
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:x]
rhs: BinaryOp(BitwiseExpr) { Nop(Identifier) [var:y] | Nop(Identifier) [var:z] }
Assign(=)
lhs: Nop(Identifier) [var:x]
rhs: BinaryOp(BitwiseExpr) { Nop(Identifier) [var:y] | Nop(Identifier) [var:z] }
Assign(=)
lhs: Nop(Identifier) [var:x]
rhs: BinaryOp(BitwiseExpr) { Nop(Identifier) [var:y] | Nop(Identifier) [var:z] }
B2 (block)
Assign(=)
lhs: Nop(Identifier) [var:x]
rhs: UnaryOp(~) { Nop(Identifier) [var:y] }
B0 (exit)
Nop(exit)
function test_int_bitwise
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:x]
rhs: BinaryOp(BitwiseExpr) { Nop(Identifier) [var:y] | Nop(Identifier) [var:z] }
Assign(=)
lhs: Nop(Identifier) [var:x]
rhs: BinaryOp(BitwiseExpr) { Nop(Identifier) [var:y] | Nop(Identifier) [var:z] }
Assign(=)
lhs: Nop(Identifier) [var:x]
rhs: BinaryOp(BitwiseExpr) { Nop(Identifier) [var:y] | Nop(Identifier) [var:z] }
B2 (block)
Assign(=)
lhs: Nop(Identifier) [var:x]
rhs: BinaryOp(BitwiseExpr) { Nop(Identifier) [var:y] | Nop(Identifier) [var:z] }
Assign(=)
lhs: Nop(Identifier) [var:x]
rhs: BinaryOp(BitwiseExpr) { Nop(Identifier) [var:y] | Nop(Identifier) [var:z] }
Assign(=)
lhs: Nop(Identifier) [var:x]
rhs: UnaryOp(~) { Nop(Identifier) [var:y] }
B0 (exit)
Nop(exit)
function test_byte_arithmetic
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:b] | Nop(Identifier) [var:c] }
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:b] | Nop(Identifier) [var:c] }
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(MulExpr) { Nop(Identifier) [var:b] | Nop(Identifier) [var:c] }
B0 (exit)
Nop(exit)
function test_ulong_arithmetic
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:b] | Nop(Identifier) [var:c] }
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(MulExpr) { Nop(Identifier) [var:b] | Nop(Identifier) [var:c] }
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(MulExpr) { Nop(Identifier) [var:b] | Nop(Literal) [const:2] }
B0 (exit)
Nop(exit)
function test_long_arithmetic
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:b] | Nop(Identifier) [var:c] }
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:b] | Nop(Identifier) [var:c] }
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(MulExpr) { Nop(Identifier) [var:b] | Nop(Identifier) [var:c] }
B0 (exit)
Nop(exit)
function test_uint_arithmetic
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:b] | Nop(Identifier) [var:c] }
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:b] | Nop(Identifier) [var:c] }
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(MulExpr) { Nop(Identifier) [var:b] | Nop(Identifier) [var:c] }
B2 (block)
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(MulExpr) { Nop(Identifier) [var:b] | Nop(Literal) [const:2] }
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(MulExpr) { Nop(Identifier) [var:b] | Nop(Literal) [const:2] }
B0 (exit)
Nop(exit)
function test_int_arithmetic
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:b] | Nop(Identifier) [var:c] }
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(AddExpr) { Nop(Identifier) [var:b] | Nop(Identifier) [var:c] }
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(MulExpr) { Nop(Identifier) [var:b] | Nop(Identifier) [var:c] }
B2 (block)
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(MulExpr) { Nop(Identifier) [var:b] | Nop(Literal) [const:2] }
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: BinaryOp(MulExpr) { Nop(Identifier) [var:b] | Nop(Literal) [const:2] }
B0 (exit)
Nop(exit)
function test_unary
B1 (entry)
Assign(=)
lhs: Nop(Identifier) [var:a]
rhs: UnaryOp(~) { Nop(Identifier) [var:b] }
B0 (exit)
Nop(exit)